

Quarkus Kafka Avro Integration Strategies

A Technical Comparison of Local Schema Source vs. Dynamic Registry Management

When integrating Quarkus microservices with Apache Kafka and Apicurio Schema Registry, architectural decisions dictate how Avro schemas (.avsc files) are managed. This document outlines the two primary operational topologies: **Static Build-Time Compilation (Schemas in Source)** and **Dynamic Runtime Resolution (No Schemas in Source)**.

1. Approach A: Maintaining .avsc Files in Source Code

This is the traditional, strongly-typed model. The .avsc schema files are treated as direct source code artifacts, stored inside the application repository, and used to generate immutable Java POJOs during the compilation lifecycle.

Workflow Mechanics

- **Authoring:** Schemas are manually defined or placed within `src/main/avro/`.
- **Code Generation:** During `mvn compile`, the Quarkus/Avro build plugins intercept these files and generate target Java classes representing the data structures.
- **Serialization:** The producer sends strongly-typed Java objects. The Apicurio SerDe reads the embedded class schema, verifies it against the registry, and publishes bytes.

Producer Configuration (application.properties)

```
mp.messaging.outgoing.orders-out.connector=smallrye-kafka
mp.messaging.outgoing.orders-out.topic=orders
mp.messaging.outgoing.orders-out.key.serializer=org.apache.kafka.common.serialization.StringSerializer
mp.messaging.outgoing.orders-out.value.serializer=io.apicurio.registry.serde.avro.AvroKafkaSerializer
mp.messaging.outgoing.orders-out.apicurio.registry.url=http://localhost:8080/apis/registry/v2
# Auto-registers the local schema if it does not exist in the remote DB
mp.messaging.outgoing.orders-out.apicurio.registry.auto-register=true
mp.messaging.outgoing.orders-out.apicurio.registry.find-latest=true
```

Consumer Configuration (application.properties)

```
mp.messaging.incoming.orders-in.connector=smallrye-kafka
mp.messaging.incoming.orders-in.topic=orders
```

```
mp.messaging.incoming.orders-  
in.key.deserializer=org.apache.kafka.common.serialization.StringDeserializer  
mp.messaging.incoming.orders-  
in.value.deserializer=io.apicurio.registry.serde.avro.AvroKafkaDeserializer  
mp.messaging.incoming.orders-in.apicurio.registry.url=http://localhost:8080/apis/registry/  
v2  
# Targets specific pre-compiled classes rather than generic records  
mp.messaging.incoming.orders-in.apicurio.registry.use-specific-avro-reader=true
```

2. Approach B: Pure Schema Registry Management (No .avsc in Source)

This is a completely decoupled, dynamic architecture. The .avsc files exist exclusively inside the Apicurio Schema Registry database. The microservice source code remains clean of structural schemas and auto-generated target files.

Workflow Mechanics

- **Governance:** Schemas are registered directly to Apicurio via CI/CD pipelines, UI, or external schema registries.
- **Dynamic Binding:** Microservices use Apache Avro's GenericRecord memory structures. The exact layout is retrieved from the registry at runtime.
- **Resolution:** The consumer reads the magic schema ID from the Kafka wire metadata, fetches the payload structure from Apicurio on-demand, and exposes fields dynamically via string lookups.

Producer Configuration (application.properties)

```
mp.messaging.outgoing.orders-out.connector=smallrye-kafka
mp.messaging.outgoing.orders-out.topic=orders
mp.messaging.outgoing.orders-
out.key.serializer=org.apache.kafka.common.serialization.StringSerializer
mp.messaging.outgoing.orders-
out.value.serializer=io.apicurio.registry.serde.avro.AvroKafkaSerializer
mp.messaging.outgoing.orders-out.apicurio.registry.url=http://localhost:8080/apis/
registry/v2
# Crucial: Map to global coordinates without local schema matching
mp.messaging.outgoing.orders-out.apicurio.registry.global-
id=io.apicurio.registry.serde.strategy.GetOrCreateIdStrategy
mp.messaging.outgoing.orders-out.apicurio.registry.avro-datum-
provider=io.apicurio.registry.serde.avro.facility.PolyglotAvroDatumProvider
```

Consumer Configuration (application.properties)

```
mp.messaging.incoming.orders-in.connector=smallrye-kafka
mp.messaging.incoming.orders-in.topic=orders
mp.messaging.incoming.orders-
in.key.deserializer=org.apache.kafka.common.serialization.StringDeserializer
mp.messaging.incoming.orders-
in.value.deserializer=io.apicurio.registry.serde.avro.AvroKafkaDeserializer
mp.messaging.incoming.orders-in.apicurio.registry.url=http://localhost:8080/apis/registry/
v2
# Explicitly force the runtime to yield GenericRecord instances
```

```
mp.messaging.incoming.orders-in.apicurio.registry.avro-datum-provider=io.apicurio.registry.serde.avro.facility.PolyglotAvroDatumProvider
```

3. Comparative Matrix

Dimension	Approach A: Schemas in Source	Approach B: Schemas in Registry Only
Type Safety	Compile-time validation. Typos catch immediately in IDE.	Runtime validation via <code>GenericRecord.get("field")</code> . Risk of runtime failures.
Deployment Coupling	High. Changes to schema require redeploying affected services.	Low. Centralized schema modifications can take effect immediately without redeploying.
Source Code State	Contains raw <code>.avsc</code> files and generated classes.	Free of generated POJOs; relies purely on dynamic client APIs.
Use Case Suitability	Enterprise systems with strict API boundaries and high domain stability.	Data pipelines, dynamic routing agents, and microservices requiring fast evolutionary updates.

Architectural Recommendation: For highly structured transactional microservices (e.g., payment engines, core banking), **Approach A** is highly recommended due to compile-time safety. For analytical layers, generic loggers, or data hubs where schema definitions change frequently without breaking application behavior, **Approach B** delivers superior operational agility.